Lab 1

Exercise 1.1 Generate graph data using Cypher

```
CREATE (university:University {name:"University of Greenwich"}) RETURN university
```




```
CREATE CONSTRAINT universityConstraint FOR (university:University) REQUIRE unive
rsity.name IS UNIQUE
```

```
MATCH (university:University {name:"University of Greenwich"})
FOREACH (name in ["Julia","Olivia","Peter","George","Katie","Rasvan","Rajesh"] | CREA
TE
(university)<-[:STUDIES_IN]-(:Student {name:name}))
```

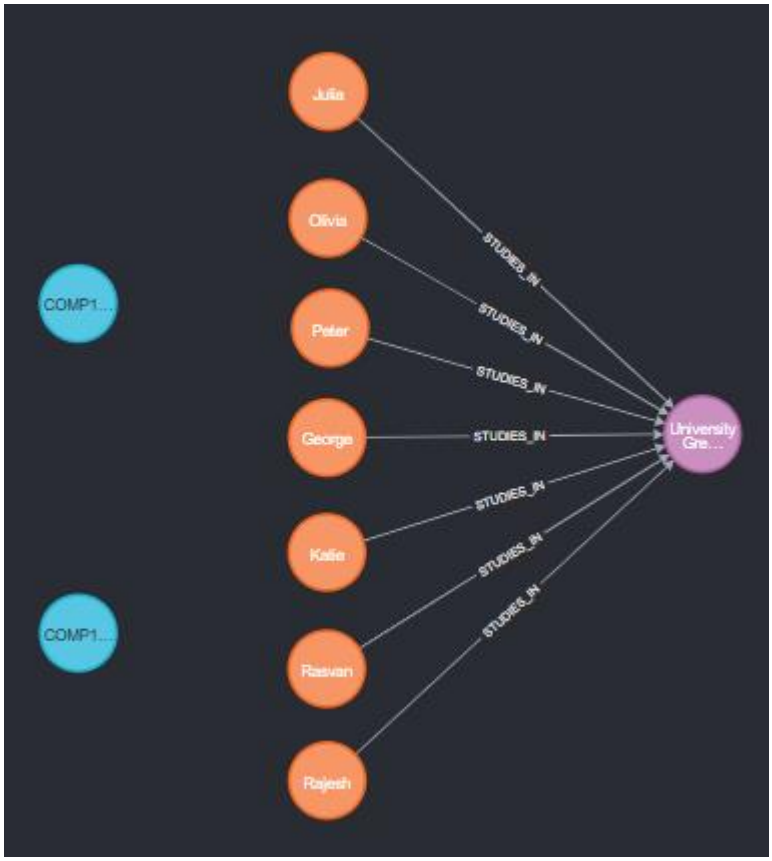
```
MATCH (university {name:"University of Greenwich"})<-[:STUDIES_IN]-
(student) RETURN
university, student
```




```
CREATE CONSTRAINT studentConstraint FOR (student:Student) REQUIRE student.name IS UNIQUE
```

```
CREATE (comp1830:Module {name:'COMP1830'})
CREATE (comp1831:Module {name:'COMP1831'})
CREATE (neo4j:Skill {name:'Ne04j'})
CREATE (ml:Skill {name:'ML'})
CREATE (python:Skill {name:'Python'})
RETURN comp1830,comp1831,neo4j,ml,python
```

```
match (n) return (n)
```




```
MATCH (comp1831:Module {name:'COMP1831'})
MATCH (student:Student {name:'Julia'})
CREATE (student)-[:ATTENDS]->(comp1831)
RETURN student, comp1831

MATCH (comp1831:Module {name:'COMP1831'})
MATCH (student:Student {name:'Olivia'})
CREATE (student)-[:ATTENDS]->(comp1831)
RETURN student, comp1831

MATCH (comp1831:Module {name:'COMP1831'})
MATCH (student:Student {name:'Peter'})
CREATE (student)-[:ATTENDS]->(comp1831)
RETURN student, comp1831

MATCH (comp1831:Module {name:'COMP1831'})
MATCH (student:Student {name:'George'})
CREATE (student)-[:ATTENDS]->(comp1831)
RETURN student, comp1831

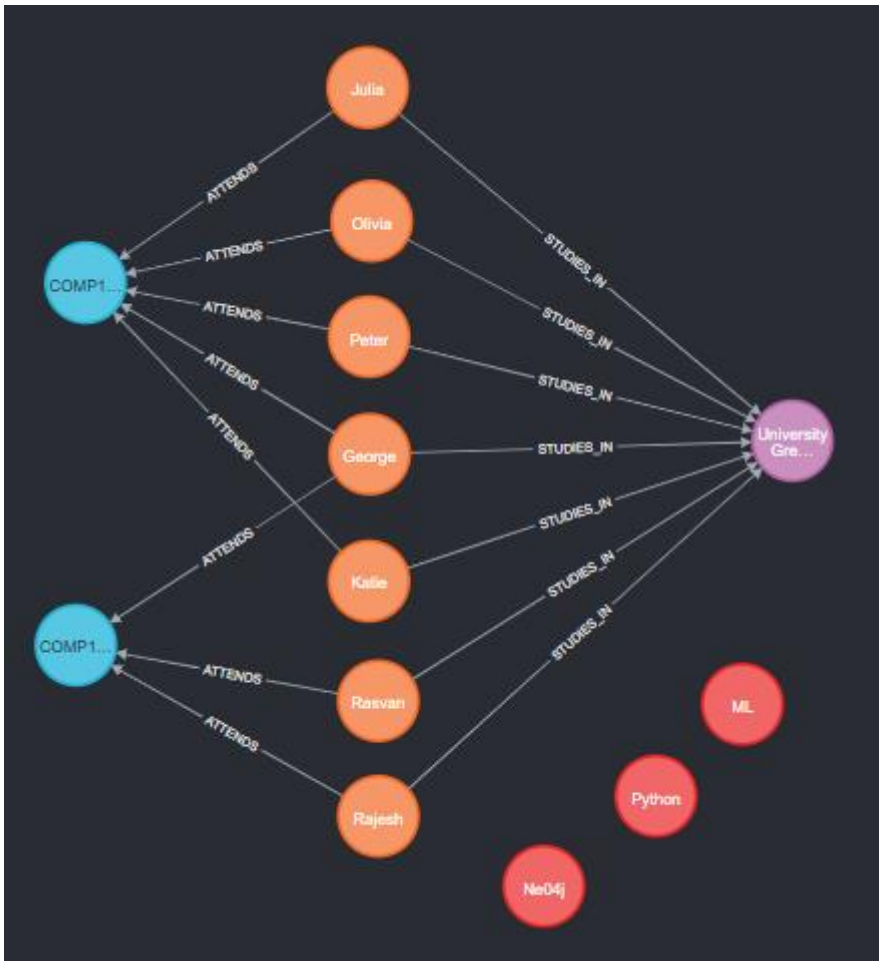
MATCH (comp1831:Module {name:'COMP1831'})
MATCH (student:Student {name:'Katie'})
CREATE (student)-[:ATTENDS]->(comp1831)
RETURN student, comp1831

MATCH (comp1830:Module {name:'COMP1830'})
```

```
MATCH (student:Student {name:'George'})
CREATE (student)-[:ATTENDS]->(comp1830)
RETURN student, comp1830

MATCH (comp1830:Module {name:'COMP1830'})
MATCH (student:Student {name:'Rasvan'})
CREATE (student)-[:ATTENDS]->(comp1830)
RETURN student, comp1830

MATCH (comp1830:Module {name:'COMP1830'})
MATCH (student:Student {name:'Rajesh'})
CREATE (student)-[:ATTENDS]->(comp1830)
RETURN student, comp1830
```

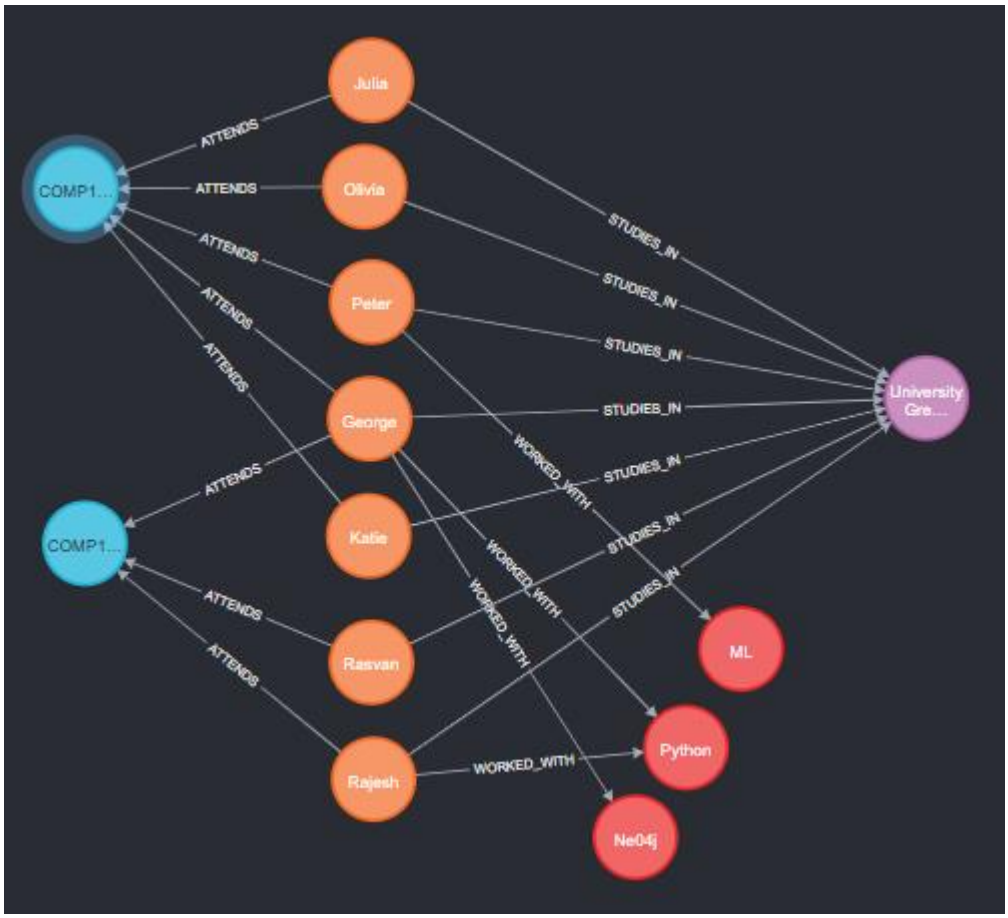



```
MATCH (ml:Skill {name:'ML'})
MATCH (student:Student {name:'Peter'})
CREATE (student)-[:WORKED_WITH]->(ml)
RETURN student, ml

MATCH (python:Skill {name:'Python'})
MATCH (student:Student {name:'George'})
CREATE (student)-[:WORKED_WITH]->(python)
RETURN student, python
```

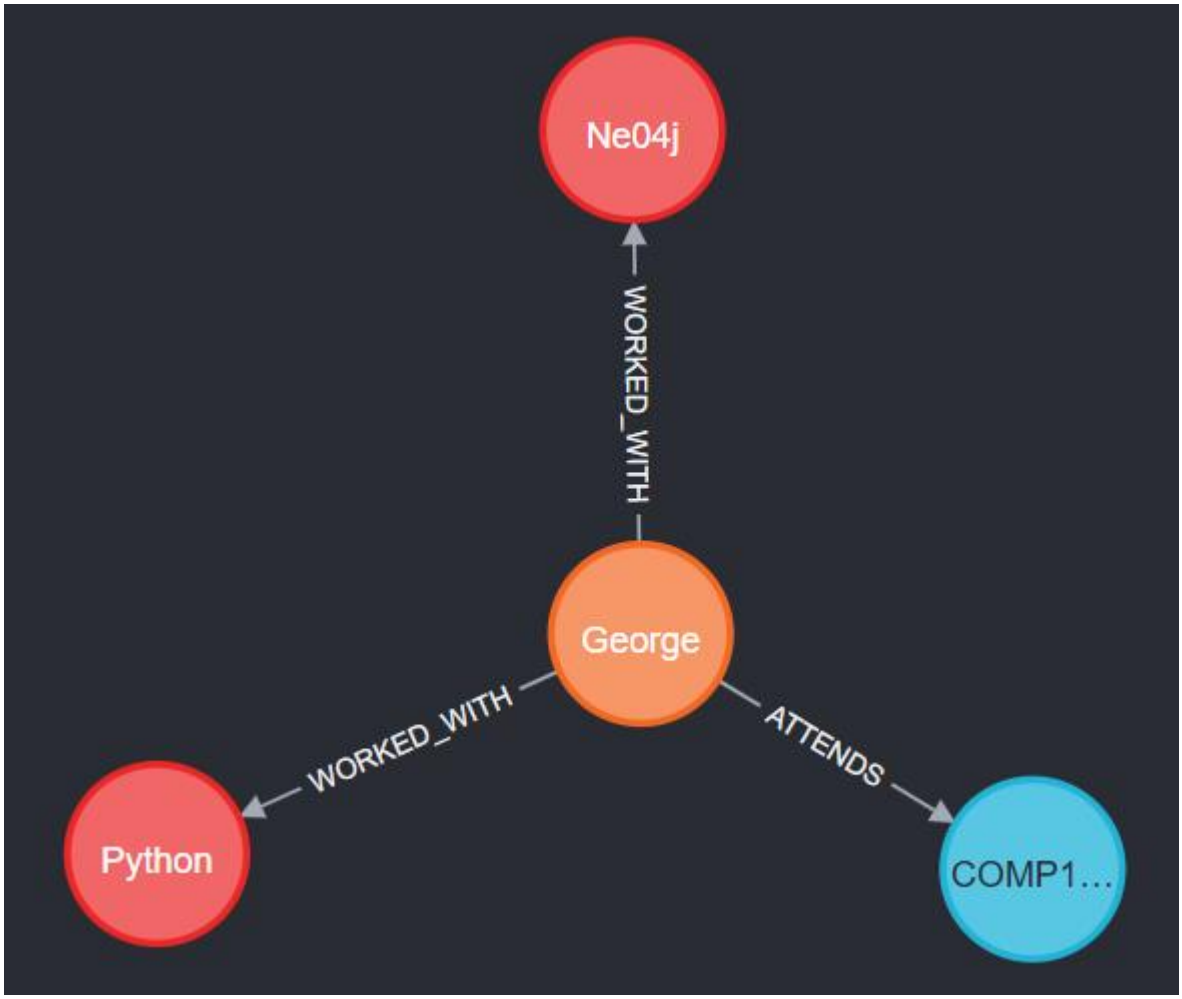
```
MATCH (neo4j:Skill {name:'Ne04j'})
MATCH (student:Student {name:'George'})
CREATE (student)-[:WORKED_WITH]->(neo4j)
RETURN student, neo4j

MATCH (python:Skill {name:'Python'})
MATCH (student:Student {name:'Rajesh'})
CREATE (student)-[:WORKED_WITH]->(python)
RETURN student, python
```



Exercise 1.2 Execute Cypher Query

```
MATCH (neo4j:Skill {name:"Ne04j"})
MATCH (python:Skill {name:"Python"})
MATCH (student)-[:ATTENDS]-
>(module:Module {name:"COMP1831"}) MATCH path1 =
shortestPath( (student)-[:WORKED_WITH*..1]-
(neo4j:Skill) ) MATCH path2 = shortestPath(
(student)-[:WORKED_WITH*..1]-(python:Skill) ) RETURN module,student,path1,path2
```



Exercise 2.1 Import CSV data using Cypher

```
CREATE CONSTRAINT personIdConstraint FOR (person:Person) REQUIRE person.id IS UNIQUE
CREATE CONSTRAINT loanIdConstraint FOR (loan:Loan) REQUIRE loan.id IS UNIQUE
```

```
LOAD CSV WITH HEADERS FROM "https://gist.githubusercontent.com/apogiatzis/237146155b85be73e25d775e487f4d8c/raw/7a440b1f4b7885ea425d5db0ce6aae041b8ea42a/persons2.csv" AS csvLine CREATE (p:Person {id:toInteger(csvLine.personId), name: csvLine.firstName, lastName: csvLine.lastName})

LOAD CSV WITH HEADERS FROM "https://gist.githubusercontent.com/apogiatzis/93927b266820ce9696b94055e896b1f9/raw/fdfdd19194b2afc1f78b01e4ddd783826b676878/loans.csv" AS csvLine MERGE (bank:Bank {name:csvLine.bank}) CREATE (loan:Loan {id: toInteger(csvLine.loanId), amount: csvLine.amount}) CREATE (loan)-[:FROM]->(bank)

LOAD CSV WITH HEADERS FROM "https://gist.githubusercontent.com/apogiatzis/dbc8f04359311257cd84e216ae8fedc4/raw/19fa315206573cab427d8b13e641c403a45852c6/status.csv" AS csvLine
MATCH (person:Person {firstName: csvLine.firstName, lastName: csvLine.lastName})
MATCH (loan:Loan {amount: toFloat(csvLine.amount)})
```

```
CREATE (status:Status {status: csvLine.status})
CREATE (person)-[:RECEIVED]->(loan)
CREATE (loan)-[:STATUS]->(status)
```

